

After we have included the dependency in the project, the next step is to find and retrieve the appropriate files:

```
bii find
```

This will create a *bii.conf* file and will download the dependencies into *bii/deps*.

To build the project, run the following command:

```
bii build
```

This will produce a folder *bin* containing the executable of the following format *user_name_project_name_main* where the project name in this case is *unit_test*; so the file will be named *unit_test_main* and if you are not logged in, the username will be replaced by the user.

Run it by:

```
./bin/user_unit_test_main
# or
bin/user_unit_test_main
```

...and the output will be similar to what follows:

```
[=====] Running 1 test from 1 test case.
[-----] Global test environment set-up.
[-----] 1 test from Sum
[ RUN      ] Sum.Normal
[ OK       ] Sum.Normal (0 ms)
[-----] 1 test from Sum (0 ms total)
[-----] Global test environment tear-down
[=====] 1 test from 1 test case ran. (0 ms total)
[ PASSED   ] 1 test.
```

So the basic workflow with BiiCode will be:

- Making a project with *bii init project_name -L*
- Including the header file in the project
- Resolving dependencies with *bii find*
- Building the project with *bii build* and running it

Optionally, you can save your work for others to reuse by publishing it with *bii publish*. Note that there is free publishing for solo open source projects—those without collaborators.

So let's try publishing it by running the following command:

```
bii publish #Run it inside the project directory
```

This will prompt you for your BiiCode account's username and password and, after authentication, your block will be publicly available under your profile. If you get a publishing error like what's shown below...

```
INFO: *****
INFO: ***** Publishing public *****
```

```
INFO: *****
```

```
ERROR: You don't have permission to publish in user/unit_
test: -1
```

...run the following:

```
bii user your_username #Replace with your username
```

...and then publish again with *bii publish*. If everything goes well, the output will be similar to what follows:

```
INFO: *****
INFO: ***** Publishing public *****
INFO: *****
INFO: Successfully published jatindhankhar/unit_test: 0
```

Your code will now be available under your profile to be reused by others just by including it as `"#include "your_username/block"`.

Enabling C++11 support

Enabling C++11 support is easy; just uncomment the following code in the *CMakeLists.txt*:

```
# > EXAMPLE: how to activate C++11
#
IF(APPLE)
    TARGET_COMPILE_OPTIONS(${BII_BLOCK_TARGET} INTERFACE
        "-std=c++11 -stdlib=libc++")
    ELSEIF (WIN32 OR UNIX)
        TARGET_COMPILE_OPTIONS(${BII_BLOCK_TARGET} INTERFACE
            "-std=c++11")
    ENDIF(APPLE)
#
```

...and C++11 features like *auto*, *range based for loops*, *smart pointers* and *thread library* are available for use.

Use cases

BiiCode can be used as an easy way to test new technologies, setting up *dev* environments and managing CI builds. We will explore other use cases and advanced usage in subsequent articles.  **END**

References

- [1] <http://docs.biiocode.com/> - BiiCode documentation
- [2] https://wiki.archlinux.org/index.php/Yaourt#Yaourt_not_using_new_AUR4_-_Yaourt_AUR4_flag
- [3] https://wiki.archlinux.org/index.php/Arch_User_Repository#AUR_4_-_AUR4_announcement

By: Jatin Dhankhar

The author loves modern C++, Ruby, JavaScript and Haskell. He can be reached at jatin@jatindhankhar.in